

Intro to Web Development – Detailed Answer Sheet

Q1.

Frontend vs Backend vs Full-stack Development

Frontend Development:

Frontend development focuses on creating the parts of a website or web application that users directly interact with. It involves designing the structure, layout, and interactivity using HTML (for content), CSS (for styling), and JavaScript (for behavior). Frameworks like React, Angular, or Vue.js are used to make dynamic and responsive interfaces.

Example: When you visit Amazon, the product images, filters, and buttons you see are part of the frontend.

Backend Development:

Backend development manages how the website works behind the scenes. It deals with server logic, database management, authentication, and APIs. Common backend technologies include Node.js, Django, Flask, Ruby on Rails, and Java Spring Boot.

Example: When you click 'Add to Cart,' the backend updates your cart details in the database and processes data securely.

Full-stack Development:

A full-stack developer works on both the frontend and backend. They understand how data flows from the user interface to the database and back.

Example: Building an entire blogging platform where one developer handles the UI, database, and server logic.

Q2.

Client-Server Model

[Client (Browser)]

|

HTTP Request (GET /products)

|

[Internet / Router / DNS]

|

[Server (Web Server + App)]

- Receives request
- App code processes, queries DB
- Produces response (HTML/JSON)

|

HTTP Response

|

[Client (Browser)] renders response

Q3.

Browser Request and Display Process

1. **URL Entry** – The user enters a web address into the browser.
2. **DNS Lookup** – The browser converts the domain name into an IP address using DNS.
3. **TCP Connection** – The browser establishes a connection with the server via TCP/IP.
4. **HTTP Request** – The browser sends an HTTP or HTTPS request to the server.
5. **Server Processing** – The server processes the request, interacts with databases, and generates a response.
6. **Resource Fetching** – The browser downloads necessary files like HTML, CSS, JS, and images.
7. **Rendering Pipeline** – The browser constructs the DOM, applies CSS, executes JavaScript, and renders the visual layout.

8. **Final Display** – The page becomes interactive and visible to the user.

Q4.

Tools for Web Development

Essential tools include:

- **Code Editors** – VS Code, Sublime Text for writing and editing code.
- **Browsers** – Chrome, Firefox, and Edge for testing and debugging.
- **Developer Tools** – Built-in browser tools for inspecting elements and debugging JavaScript.
- **Version Control** – Git and GitHub for code tracking and collaboration.
- **Package Managers** – npm or yarn for managing libraries.
- **Databases** – MySQL, MongoDB for storing data.
- **Local Servers** – XAMPP, WAMP, or Node.js for local testing.

Q5.

Web Server and Examples

A web server stores, processes, and delivers web content to users over the Internet using HTTP or HTTPS protocols. It interprets client requests, serves files, or communicates with backend applications.

Examples:

- **Apache** – Open-source, widely used, reliable for hosting dynamic and static content.
- **Nginx** – Efficient for handling high traffic and load balancing.
- **IIS** – Microsoft's web server for Windows-based systems.
- **Node.js** – Handles asynchronous server operations using JavaScript.
- **Caddy** – Modern server with automatic HTTPS.

Q6.

Roles in a Web Project

1. **Frontend Developer** – Designs and develops the interface, ensuring responsiveness and cross-browser compatibility.
2. **Backend Developer** – Builds APIs, manages databases, handles security, and ensures server-side performance.
3. **Database Administrator (DBA)** – Maintains data integrity, backups, and query optimization.
4. **UI/UX Designer** – Creates wireframes and prototypes focusing on user experience.
5. **Project Manager** – Coordinates between team members, sets deadlines, and ensures

progress.

Q8.

Static vs Dynamic Websites

- **Static Websites:** Built with fixed HTML, CSS, and JavaScript files. They don't change unless manually edited.

Example: Personal portfolio or company landing page.

Advantages: Fast, simple, and secure.

Disadvantages: Difficult to update or personalize content.

- **Dynamic Websites:** Generate content dynamically using server-side languages and databases.

Example: Facebook, blogs, or e-commerce platforms.

Advantages: Easier to maintain, supports user interaction, and personalized content.

Disadvantages: Requires backend setup and can be slower without optimization.

Q9.

Web Browsers and Rendering Engines

Each browser uses a rendering engine that determines how HTML, CSS, and JavaScript are processed and displayed:

- **Google Chrome:** Blink engine – fast and secure.
- **Mozilla Firefox:** Gecko engine – supports advanced web standards.
- **Microsoft Edge:** Blink engine – same as Chrome for compatibility.
- **Safari:** WebKit engine – optimized for Apple devices.
- **Opera:** Blink engine – lightweight with additional tools.

Rendering engines affect page layout, animations, performance, and how code is interpreted.

Q10.

Basic Web Architecture Flow

The web architecture defines how different components communicate to deliver content to users.

1. **Client Browser:** Sends HTTP requests (GET/POST) when a user interacts with a website.
2. **Web Server:** Handles incoming requests and forwards them to the application server.
3. **Application Server:** Processes business logic, runs scripts, and interacts with

databases.

4. ****Database:**** Stores and retrieves structured data as per the application's needs.
5. ****Response Cycle:**** The data flows back through the application and web servers to the client for display.

Example: When you submit a login form, the request goes to the server, credentials are checked in the database, and the response shows whether login succeeded or failed.